

SEN381 - MILESTONE 2: ARCHITECTURE, TECHNOLOGY & INITIAL DESIGN BASELINE

PED Version 2.0

Team Members:

Aidan Smith - 601819
Enrique de Sousa - 603110
Michael-John Robinson - 602310

1) PROBLEM & BUSINESS NEED / STAKEHOLDER ANALYSIS

- 1.1) Problem and business need
- 1.2) Stakeholder analysis
- 1.3) Linked requirement examples

2) SCOPE BASELINE & CONSTRAINTS

- 2.1) Scope baseline
- 2.2) Constraints
- 2.3) Constraint interaction

3) REQUIREMENTS & ACCEPTANCE CRITERIA

- 3.1) Functional requirements
- 3.2) Non-functional requirements
- 3.3) Prioritisation note

4) REQUIREMENTS TRACEABILITY MATRIX (RTM)

5) ASRs AND QUALITY DRIVERS

6) RISK REGISTER & FORWARD ENGINEERING CONSIDERATIONS

- 6.1) Initial Risk Register
- 6.2) Forward Engineering Considerations

7) ARCHITECTURE DECISION AND DIAGRAMS

8) DATA AND PERSISTENCE BASELINE

- 8.1) Entities, ownership and lifecycle
- 8.2) Initial data model
- 8.3) Persistence model, SPOF and recovery
- 8.4) Transactions, validation and concurrency

9) TECHNOLOGY-STACK DECISION

10) INITIAL DESIGN DECISIONS

11) INITIAL API / INTEGRATION DECISIONS

12) DEPLOYMENT COMPATIBILITY

13) GITHUB GOVERNANCE, TEAM ACCOUNTABILITY & AI USAGE REGISTER

- 13.1) Repository governance evidence
- 13.2) Team working agreement
- 13.3) AI Usage Register

14) ENGINEERING DECISION LOG

15) PED QUALITY, DOCUMENT CONTROL & BASELINE SIGN-OFF

- 15.1) Document control
- 15.2) Baseline sign-off

16) REFERENCES

1) PROBLEM & BUSINESS NEED / STAKEHOLDER ANALYSIS

1.1) Problem and business need

The organisation currently has a disjointed approach to service requests. They are made by email, on WhatsApp, on spreadsheets, and on paper. This results in issues of duplicated and lost requests, lack of clarity over ownership, lack of visibility for requesters, inconsistent prioritisation, and poor accountability. Management is also less effective in getting reliable information on outstanding, overdue and completed requests, and sensitive information may be handled insecurely through informal channels. The proposed solution is a controlled digital environment that will ensure that there is one single record for every service request with a single and traceable record for all service requests from submission to closure. This will help the business to create value by enhancing the efficiency of operations, transparency, accountability and information management. Requests would be more visible to the requester, staff would have a better sense of ownership and prioritisation, management would have more accurate information for them to monitor and make informed decisions. The solution should be usable, secure, maintainable and financially sustainable to the organisation and deliver these benefits.

1.2) Stakeholder analysis

Stakeholder: Requesters

Requesters, people from the community who place a “service request”, need to have a convenient and easy to use way to report an issue and get an acknowledgement that their request was received. They also want to see the status of the request, the staff member assigned to the request, progress, delays and final resolution. They are primarily interested in timely and effective service, and have a relatively low influence on the system compared to management and staff. There can be a conflict if the elements are expecting a quick reply or a priority to be given to the request, while the staff and management is prioritising the requests according to the urgency, resources and organisation policies.

Stakeholder: Service Staff

Service staff should be responsible for receiving, reviewing, assigning, prioritising and resolving service requests. They want a system where they can access information around outstanding work, priorities, deadlines, request details and ownership. They have strong influence and interest in the system as they will be using the system regularly and their work will be directly affected by the system. Staff might not want a more flexible and readily used system while management may want more control, documentation and reporting. Some conflict may also occur if the system is seen as adding to administration or as involving monitoring of staff performance.

Stakeholder: Management

Management needs reliable, real-time information about the number and types of requests, workloads, overdue issues, resolution times and overall service performance. Their involvement in the project is great as they make decisions related to priorities, resources, policies and system adoption. Management has a keen interest in accountability, reporting and operational efficiency. However, their expectations may be different from those of management when management requires a high level of tracking and reporting and staff may feel that they are being monitored too much with extra administrative requirements or lack flexibility in the way that they work.

Stakeholder: IT/System Administrators

IT staff or system administrators require a solution that is secure, maintainable, scalable and manageable within the technical resources of the organisation. They handle user access, system configuration, data protection, backup and troubleshooting, and may even be responsible for integration with existing systems. They have a strong influence since technical choices can have long-term consequences for the sustainability of the platform. A potential conflict exists between IT's need for security, governance and controlled changes and the organisation's desire for a solution that is inexpensive, simple and quick to implement.

Stakeholder: Organisation/Business Owners

Business owners or senior decision makers in the organisation are interested in improvements that are measurable and not an ongoing financial or operational burden. They must have the solution that will meet the goals of the organisation to implement and sustain. They are powerful in that they are able to approve requirements, budgets, and major changes. The business value of their expectation of a cost-effective solution that can be relatively straightforward may clash with IT or management needs for more sophisticated security, reporting, integration and functionality which necessitates a balance between value and complexity and cost of the project team.

1.3) Linked requirement examples

An example of a requirement that traces directly back to one of the aforementioned stakeholder needs is that the requester must be able to see if the request has been received, assigned, delayed, resolved or closed. Thus, the system should enable the requesters to track the current status of their requests during the request lifecycle. This requirement goes directly to the current situation where the requester doesn't know what is happening, and they have to reach out to staff in other ways to find out.

2) SCOPE BASELINE & CONSTRAINTS

2.1) Scope baseline

In scope:

- Submitting, categorising, viewing, and tracking service requests.
- Viewing request history and receiving feedback on request status changes.
- Allowing authorised staff to search, filter, view, assign, update, and resolve service requests.
- Recording staff actions, comments, and resolution information.
- Providing management with service activity information including open, overdue, resolved, and closed requests.
- Reporting on requests by category, status, and other justified dimensions.
- Supporting accountability and service performance analysis.

Out of scope:

- Features that are not directly related to the core service-request process.
- Additional functionality that cannot be justified by clear stakeholder value.
- Features that would introduce unnecessary scope, security, cost, schedule, quality or maintenance risks.

Deferred / future scope:

- Additional features proposed by stakeholders that require further analysis before implementation.
- Advanced functionality that cannot be completed within the initial project scope without compromising quality.
- Enhancements identified after the core system has been successfully implemented and tested.

In defense of advanced reporting and analysis being deferred, the primary emphasis of the initial system must be on the core service-request process. At this stage it would be difficult to justify the complexity of introducing more advanced dashboards and performance analytics since they are not necessarily a vital part of the minimum business requirements. The feature may be re-enabled when the core system is stable enough and enough data about the service-request system is gathered. It may be rethought if management determines that a more detailed performance evaluation is warranted and the extra development time, effort, cost, security and testing requirements can be justified within the project.

2.2) Constraints

Constraint type: Scope

- Description: The system should be targeted towards the minimum business capabilities: submission of service requests, tracking, assignment, and resolution of service requests. Additional features need to be warranted and not unnecessarily added to the project.
- Engineering implication: The team has to focus on core functionality and scope creep has to be avoided. Requirements should be baselined as early as possible with further features postponed if there is insufficient time or money.

Constraint type: Schedule

- Description: The project will have a fixed project deadline (19 October 2026) for when it will need to be designed, implemented and tested.
- Engineering implication: Essential requirements should be prioritised for development. The team should work incrementally and utilise the provided milestones so that a system is developed and working by the deadline.

Constraint type: Cost/Resources

- Description: The project has limited resources in terms of team members, development time, available technology and possibly limited access to paid services or infrastructure.
- Engineering implication: It is important to the team to adhere to the use of technologies and services that are readily available and to stay away from the need of using external technologies or expensive infrastructure. The features to be included should be based on the value that they add to development effort.

Constraint type: Quality

- Description: The system needs to be reliable, usable and maintainable while correctly handling service requests and status changes. A larger system should not be used if it does not yield an adequate or thoroughly tested solution.
- Engineering implication: Requirements must be testable and adequate time must be provided for the integration, validation and user testing. The team should strive for a smaller, consistent system rather than a bigger system that has defects.

Constraint type: Security

- Description: Sensitive information may be included in a service request and only be available to authorised users. Various users, such as requesters, staff and management could need different levels of access.
- Engineering implication: The system must implement authentication and role-based access control, protect stored and transmitted data, validate user input and prevent unauthorised access. Security must be built in from the beginning of the design process.

2.3) Constraint interaction

One of the major trade-offs is the relationship between schedule, scope and quality. If there is a time constraint on the project and a lot of extra features are added onto, the development will take more time and there will be less time to test and fix any defects. This may lead to lower overall quality and reliability of the system. In order to control this, the team must focus on the features and functions that are essential to the business and postpone any features that are not in order to give sufficient time to test the core system and ensure that the core is completed to a high standard.

3) REQUIREMENTS & ACCEPTANCE CRITERIA

3.1) Functional requirements

ID	Stakeholder	Priority	Acceptance criteria
FR-001: Submit a new informative service request	Requester - Member requesting service	High	Requester able to submit requests including fields. The system rejects requests without mandatory information. The system generates a unique ID.
FR-002: Categorise requests	Requester - Member requesting service	High	Requests have controlled category/s which are mandatory. The category is filterable
FR-003: View status of submitted requests	Requester - Member informed on status	High	Requesters can view the current status (e.g. Submitted, Assigned, In Progress, Resolved, Closed) of any request they submitted, from a request list or detail view.
FR-004: A history/list of previous submitted requests	Requester - Member interested in previous requests	Medium	Previous requests are viewable with the relevant information such as resolved, unresolved, etc.
FR-005: Receive meaningful feedback when status is updated	Requester - Member informed on request	High	The requester receives feedback about the request when it has been accepted, rejected, completed, etc. Staff are able to provide feedback
FR-006: View service requests relevant to authorised staff	Staff - Member viewing requests	High	Staff can only view requests assigned to their role/department; attempting to open a request outside their role returns access denied or is not listed.
FR-007: Search, filter or sort requests	Staff - Member viewing requests	Medium	Staff are able to scan through requests based on criteria such as role, ID, service type, etc
FR-008: View full request details	Staff - Member viewing requests	High	Staff can view requester details, category, description, submission date, status history, and any attached comments/actions for a given

ID	Stakeholder	Priority	Acceptance criteria
			request.
FR-009 Assign/accept responsibility for request	Staff - Member taking request	High	Staff can be assigned a request and also accept a request. Requests cannot be double assigned. Staff are able to see what requests they are responsible for
FR-010 Update request status	Staff - Member doing request	High	Staff is able to change the status of the request based on the status flow. The status must be updated on the requester's side too.
FR-011: Record relevant details on request resolution process	Staff - Member doing request	Medium	Staff are able to record the progression of the request resolution. Details must be stored for future references and learning.
FR-012: Resolve/close requests where authorised	Staff - Member doing request	High	Closed requests are moved to a history/archive view but remain retrievable for audit purposes; they are removed from the active/open requests list.
FR-013: View service activity information	Management - Member managing staff activity	High	Management is able to see the progression of requests being done by staff such as activity and completeness.
FR-014: Identify the status of requests	Management - Member viewing progress of requests	High	Management is able to see the status of requests, whether it's been assigned, opened, resolved, overdue, etc.
FR-015: View request info based on criteria	Management - Member viewing requests	Medium	Management can filter requests by category, status, etc, to see certain information
FR-016: Access information to support accountability and service-performance analysis	Management - Member monitoring staff performance	High	Management is able to see who is assigned to requests, the requests progress and the status of the request in terms of performance and quality.

3.2) Non-functional requirements

ID	Stakeholder	Priority	Acceptance criteria
NFR-001: Response time (Performance)	All users - Members submitting/viewing requests	High	95% of standard actions (submit, view status, search) complete within 2 seconds under normal load (up to 50 concurrent users).
NFR-002: Authenticated access (Security)	Staff/Management - Members accessing controlled functions	High	100% of staff/management functions require valid login. Unauthenticated access attempts are rejected and logged.
NFR-003: Role-based access control (Security)	Staff - Members with restricted role access	High	Staff cannot view or act on requests outside their authorised role/category. Verified through access-control testing with no exceptions.
NFR-004: Data protection (Security)	Requester - Member submitting personal/request information	High	All data in transit uses HTTPS/TLS. Passwords/credentials are hashed and never stored in plaintext. Verified via security review.
NFR-005: System availability (Reliability)	All users - Members relying on system access	Medium	The system maintains at least 99% uptime during defined operational hours. Downtime is logged with cause recorded.
NFR-006 Browser compatibility	All users - Members accessing via standard browsers	Low	The system is functional on at least two current major browsers (e.g. Chrome, Edge/Firefox) at common screen sizes.
NFR-007 Ease of use (Usability)	Requester/Staff - Member with no prior training	Medium	A first-time user can submit a request or update a status within 3 attempts without external help. Verified via usability test (3-5 participants).
NFR-008 Data integrity on failure (Reliability)	All users - Members relying on submitted data being preserved	Medium	A backup/recovery mechanism exists. Simulated failure/restore test confirms no submitted request data is lost.

NFR-009 Maintainability	Development team - Members maintaining/extending the system	Medium	Code follows an agreed structure/style standard, includes documentation for key modules, and passes static analysis with no critical issues.
NFR-010 Status change traceability (Auditability)	Management - Member requiring accountability evidence	High	Every status change is logged with user ID, timestamp, and previous/new status. 100% retrievable via audit/history view.

3.3) Prioritisation note

Requirements were rated **High** where they are essential to the core business need identified in the Project Brief, providing a reliable, traceable way to submit, manage, monitor and report on service requests, or where their omission would create a direct security or accountability gap.

Requirements were rated **Medium** where they materially improve usability, oversight, or operational quality, but are not strictly required for the system to operate in a controlled and functional manner.

Requirements were rated **Low** where they provide additional convenience or polish without affecting core capability, business value, or risk exposure.

4) REQUIREMENTS TRACEABILITY MATRIX (RTM)

Req ID & requirement	Source / priority	ASR / quality driver	Architecture responsibility	Data / persistence impact	Design / interface decision	Technology decision	Implementation evidence	Verification evidence	Status	ADR / change / risk ref.
FR-001 Submit service request	Requester / High	NFR-004, NFR-001, NFR-010	Requests module	ServiceRequest + first StatusHistory row; mandatory fields NOT NULL; unique ReferenceNo	Single synchronous transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	PR #3 (merged): ServiceRequestsController.Create, Create.cshtml, InitialCreate migration. Partial – ReferenceNo and StatusHistory not yet built	Planned: valid submit → unique ID; missing field → rejected, nothing saved	In Development	R-005
FR-002 Categorise requests	Requester / High	–	Requests module	Category reference table; mandatory FK	Controlled lookup + dropdown; server-side category check	ASP.NET Core 8 MVC; EF Core + SQL Server	PR #3, commit 797642e: Category.cs, CategoryId FK, AddCategoryLookup migration, dropdown in Create.cshtml	Planned: no category → rejected; filter by category	Implemented	D-001
FR-003 View status	Requester / High	NFR-001, NFR-003	Requests module	CurrentStatus on ServiceRequest; query	State pattern	ASP.NET Core 8 MVC; EF Core + SQL Server	Partial: Details view shows status (PR #3); requester scoping not yet built	Planned: requester sees own requests only	In Development	R-003

				scoped to requester						
FR-004 Request history	Requester / Medium	NFR-001	Requests module	No hard deletes; closed requests retained	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: resolved + unresolved listed	Approved	FEC #4
FR-005 Feedback on status update	Requester, Staff / High	NFR-010	Assignment → Notification (in-process)	RequestComment (Feedback, visible to requester)	In-process INotificationService, same transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: status change creates requester-visible feedback; failure rolls back the status change	Approved	D-003
FR-006 Staff see authorised requests only	Staff / High	NFR-003, NFR-002	Users/Auth + Security modules	StaffCategory Scope filters every staff query	Single authorisation point in service layer	ASP.NET Core Identity + policy authorisation	Not Yet Implemented	Planned: out-of-scope request denied / not listed	Approved	R-002; FEC #1
FR-007 Search / filter / sort	Staff / Medium	NFR-001	Requests module	Indexes on status, category, assignee, date	DI repositories –	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: each criterion: ≤ 2 s	Approved	FEC #5
FR-008 Full request details	Staff / High	NFR-003, NFR-004	Requests module	Joins request, requester, history, comments	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: all fields shown; unauthorised denied	Approved	R-002
FR-009 Assign / accept	Staff / High	NFR-010, NFR-001	Assignment module (RequestWorkflowService)	Single AssignedStaffId; optimistic concurrency (RowVersion)	Optimistic locking	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: two concurrent accepts → one succeeds	Approved	D-002; R-003
FR-010 Update status	Staff / High	NFR-010, NFR-003	Assignment module (RequestWorkflowService)	Status update + StatusHistory in one transaction	State pattern – D-005; feedback via	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: invalid transition rejected; history row written	Approved	R-003
FR-011 Record resolution details	Staff / Medium	NFR-010	Assignment module (RequestWorkflowService)	RequestComment (ResolutionNote), append-only	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: notes retrievable after closure	Approved	FEC #4
FR-012 Resolve / close	Staff / High	NFR-003, NFR-010	Assignment module (RequestWorkflowService)	Status → Closed; archived, never deleted	State pattern	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: leaves active list, stays in history	Approved	R-002; R-003
FR-013 Service activity info	Management / High	NFR-010	Reporting (read-only queries)	Read-only queries over requests + history	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: matches seeded data	Approved	FEC #6

FR-014 Status incl. overdue	Managem ent / High	NFR-001	Reporting (read-only queries)	"Overdue" needs a due-date rule – not yet defined	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned once rule defined	Approv ed – clarifica tion open	R-003
FR-015 Filter by criteria	Managem ent / Medium	NFR-001	Reporting (read-only queries)	Reuses FR-007 indexes	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: correct subset returned	Approv ed	FEC #5
FR-016 Accountability info	Managem ent / High	NFR-010	Reporting (read-only queries)	Depends on complete StatusHistory	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: assignee, progress, status shown	Approv ed	FEC #6
NFR-001 Response time	All / High	ASR	Cross-cutting	Indexing; single DB is bottleneck	Modular monolith, in-process calls	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: 50 users, 95% ≤ 2 s	Approv ed	FEC #5
NFR-002 Authentication	Staff/Mgm t / High	Security	Users/Auth + Security modules	User credentials; failed logins logged	IPasswordHas her / Identity	ASP.NET Core Identity	Not Yet Implemented	Planned: unauthenticate d calls rejected + logged	Approv ed	R-002
NFR-003 RBAC	Staff / High	ASR	Users/Auth + Security modules	Role + StaffCategory Scope	Single authorisation point in service layer	ASP.NET Core Identity + policy authorisation	Not Yet Implemented	Planned: role × category test matrix	Approv ed	R-002; FEC #1
NFR-004 Data protection	Requester / High	ASR	Users/Auth + Security modules	PasswordHas h only; personal data sensitive	IPasswordHas her	Identity password hashing; HTTPS via Azure App Service	Not Yet Implemented	Planned: security review; no plaintext passwords	Approv ed	R-002
NFR-005 Availability	All / Medium	Reliability	Azure App Service	Single DB is a SPOF	No external dependency in the status path	Azure App Service F1	Not Yet Implemented	Planned: uptime log	Approv ed	R-004; R-005
NFR-006 Browser compatibility	All / Low	–	MVC Razor views	None	Razor MVC views	Razor views + Bootstrap	Not Yet Implemented	Planned: Chrome + Edge/Firefox	Approv ed	FEC #3
NFR-007 Ease of use	Requester/ Staff / Medium	–	MVC Razor views	None	Razor MVC views	Razor views + Bootstrap	Not Yet Implemented	Planned: usability test (3–5 users)	Approv ed	–
NFR-008 Data integrity on failure	All / Medium	Reliability	Data-access layer	Backup/restor e baseline	Backup before each migration	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: restore test, no data lost	Approv ed	D-004; R-005
NFR-009 Maintainability	Dev team / Medium	Quality	Cross-cutting	Version-contr olled EF migrations	DI	ASP.NET Core 8 MVC; EF Core + SQL Server	EF Core migrations version-controlled in repo (PR #3)	Planned: static analysis; PR review	In Develop ment	R-001; FEC #2

NFR-010 Status traceability	Managem ent / High	ASR	Audit module	Append-only StatusHistory, same transaction as change	Single synchronous transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	Not Yet Implemented	Planned: 100% of changes have history row	Approv ed	D-003; D-004; R-003
--	-----------------------	-----	--------------	---	--------------------------------------	---	------------------------	--	--------------	---------------------------

Which columns progressed since M1

The M1 RTM only had ID, source and summary, with design, implementation and test marked as future. M2 adds the ASR link, the module that owns each requirement, data impact, design decisions, the technology stack, a planned verification per acceptance criterion, and ADR/risk links. Implementation evidence now exists for FR-001, FR-002, FR-003 and NFR-009 (PR #3, merged into main on 30/09/2026); all other rows stay "Not Yet Implemented" and will be updated as feature branches merge.

FR-001 end-to-end trace (continued from M1)

FR-001 → ASRs NFR-004/NFR-010 → Requests module → ServiceRequest + first StatusHistory row in one synchronous transaction → validation in UI, service and database layers → ASP.NET Core 8 MVC with EF Core + SQL Server → application artefact: ServiceRequestsController.Create, Create.cshtml and InitialCreate migration (PR #3); ReferenceNo and StatusHistory still to be built → verification: Planned (valid submit returns unique ID; missing field rejected with nothing save).

5) ASRs AND QUALITY DRIVERS

ASR candidate: NFR-003 Role-based access control (Security, High priority)

- Already flagged in M1 Forward Engineering Consideration #1 (RBAC) as influencing "architecture and technology-stack selection"
- Measurable/testable expectation (from M1): Staff cannot view/act on requests outside authorised role/category, verified through access-control testing with no exceptions
- How it influences architecture/data/technology/design: RBAC necessitates architecture and technology that provides for centralised authentication and authorisation. Data model and design should restrict access to roles and categories at the back end/API level.

ASR candidate: NFR-001 Response time (Performance, High priority)

- Measurable/testable expectation (from M1): 95% of standard actions complete within 2 seconds under normal load (up to 50 concurrent users)
- Connects to Forward Engineering Consideration #5 (Scalability under concurrent use)
- How it influences architecture/data/technology/design: The architecture needs to be scalable, with optimised data access and services, to ensure it can handle the processing and concurrent users efficiently.

ASR candidate: NFR-010 Status change traceability (Auditability, High priority)

- Measurable/testable expectation (from M1): Every status change logged with user ID, timestamp, previous/new status; 100% retrievable via audit/history view

- How it influences architecture/data/technology/design: Needs an audit-log data structure and backend mechanisms to capture all the status changes with whom, when and the history of status.

ASR candidate: NFR-004 Data protection (Security, High priority)

- Measurable/testable expectation (from M1): HTTPS/TLS in transit, hashed credentials, verified via security review
- How it influences architecture/data/technology/design: Needs HTTPS/TLS, secure password hashing, and security conscious technology and architecture decisions to secure data in transit and at rest.

6) RISK REGISTER & FORWARD ENGINEERING CONSIDERATIONS

6.1) Initial Risk Register

Organisations building e-service systems frequently struggle with poor risk practices, particularly under budget and schedule pressure. This gap is a recurring contributor to project failure (Pilliang & Munawar, 2022). Vague and generic risk entries provide little actionable value whereas effective registers have a clear cause with measurable impact and an owner who can act on it.

Risk ID	Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
R-001	Two-reviewer PR approval becomes a bottleneck given only 3 team members, especially near deadlines when availability is uneven	Small team size means every PR needs buy-in from both remaining members, who may be busy with their own sections	Medium	Medium	Medium	Agree a team working-agreement turnaround time for reviews stagger individual workstreams so PRs don't all land at once	If a reviewer is unavailable, escalate to the third member or delay merge rather than bypass the two-approval rule	Aidan Smith	Mitigated
R-002	Sensitive request data is handled without adequate access control, creating a security exposure	RBAC not yet designed; requirements may under-specify who can view what	Medium	High	High	Capture data-sensitivity and access-control needs explicitly as NFRs now	If a gap is found post-implementation, restrict the affected view immediately	Michael -John Robinson	Open

							and treat as a formal defect		
R-003	Core request-status workflow is under-specified, causing rework once management views are designed in M2	Status-transition rules not fully defined at requirements stage	Medium	Medium	Medium	Explicitly define allowed status transitions as part of FR acceptance criteria before workflow design in M2	If a missing transition is discovered later, treat as a controlled requirements change, not an ad hoc code fix	Michael -John Robinson	Mitigated
R-004	A requirement or NFR quietly assumes deployment infrastructure the team cannot actually afford or access	Cost constraint not cross-checked against requirements as they are written	Low	Medium	Medium	Research realistic free hosting options before finalising NFRs	If discovered late, scale back the specific requirement rather than the whole architecture	Enrique de Sousa	Open, mitigation pending validation
R-005	Database single point of failure / data loss	All request and audit data sits in one database; an outage stops the service and loss or corruption destroys accountability evidence	Low	High	Medium	Daily off-host backups, a backup before each migration	Restore latest backup (RPO $\leq$ 24 h, RTO $\leq$ 4 working hours)	Michael -John Robinson	Open

6.2) Forward Engineering Considerations

Design choices are often reasonable at the moment they're made but measurably reduce a system's maintainability and scalability as the system grows if the implications are not considered early (Verdecchia, Kruchten, Lago, & Malavolta, 2021).

1. Authentication & role-based access control (RBAC) [Addressed: Section 7/9]

- Why it matters now: Three distinct user types need different visibility and permissions, shaping how requirements and data models are framed.
- Later Influences: Architecture and technology-stack selection.
- Missing Information: Exact permission boundaries per role
- Postponement Risk: Retrofitting access control after implementation typically requires touching most of the codebase.

2. Automated testing & CI [Deferred to M3]

- Why it matters now: If requirements aren't written in testable, acceptance-criteria form now, retrofitting testability later is costly.
- Later Influences: CI pipeline design and test strategy.

- Missing Information: Which requirements are highest-risk and therefore need the most test coverage.
- Postponement Risk: Requirements without measurable acceptance criteria are difficult to verify automatically, forcing manual testing.

3. Deployment environment & platform compatibility [Addressed: Section 12]

- Why it matters now: Cost/schedule constraints mean the team should understand hosting limitations before committing to requirements that assume unavailable infrastructure.
- Later Influences: Technology-stack decision and staging deployment.
- Missing Information: What free/low-cost hosting options are actually viable for the stack choices.
- Postponement Risk: A requirement or NFR could quietly assume infrastructure the team cannot deploy affordably.

4. Data backup & recovery [Addressed: Section 8]

- Why it matters now: Service-request records are the core asset of the system. Loss of data has direct consequences for every stakeholder group.
- Later Influences: Data/persistence design and operational readiness review.
- Missing Information: What recovery point/time expectations are realistic.
- Postponement Risk: No backup strategy discovered only after a data-loss incident, rather than factored in from the start.

5. Scalability under concurrent use [Addressed: Section 5/7]

- Why it matters now: Staff and management views involve simultaneous access to shared request data.
- Later Influences: Architecture and performance testing.
- Missing Information: Realistic expected concurrent user counts.
- Postponement Risk: Performance NFRs left vague now become impossible to test meaningfully.

6. Observability & monitoring [Deferred to M3]

- Why it matters now: Management's oversight capabilities depend on the system being able to report on its own state.
- Later Influences: Operational readiness and incident response.
- Missing Information: What minimum monitoring is realistic to implement within constraints.
- Postponement Risk: It will become hard to know if the system is degrading without planned logging

7) ARCHITECTURE DECISION AND DIAGRAMS

Realistic architecture alternatives considered:

- Alternative A: Modular monolith. One deployable backend exposing a REST API, organised internally into modules. A web front end serves all three user types, and a single relational database holds the data.
- Alternative B: Microservices. Separate services for authentication, request management, assignment, notifications and audit, each with its own data store, communicating via APIs or a message broker behind an API gateway.
- Alternative C: Serverless/Backend-as-a-service (BaaS). Managed authentication, a hosted database and cloud functions, with a thin client. Little custom server code is written.
- Alternative D: Separate applications per user type. Independent Requester, Staff and Management applications sharing a database or duplicating logic.

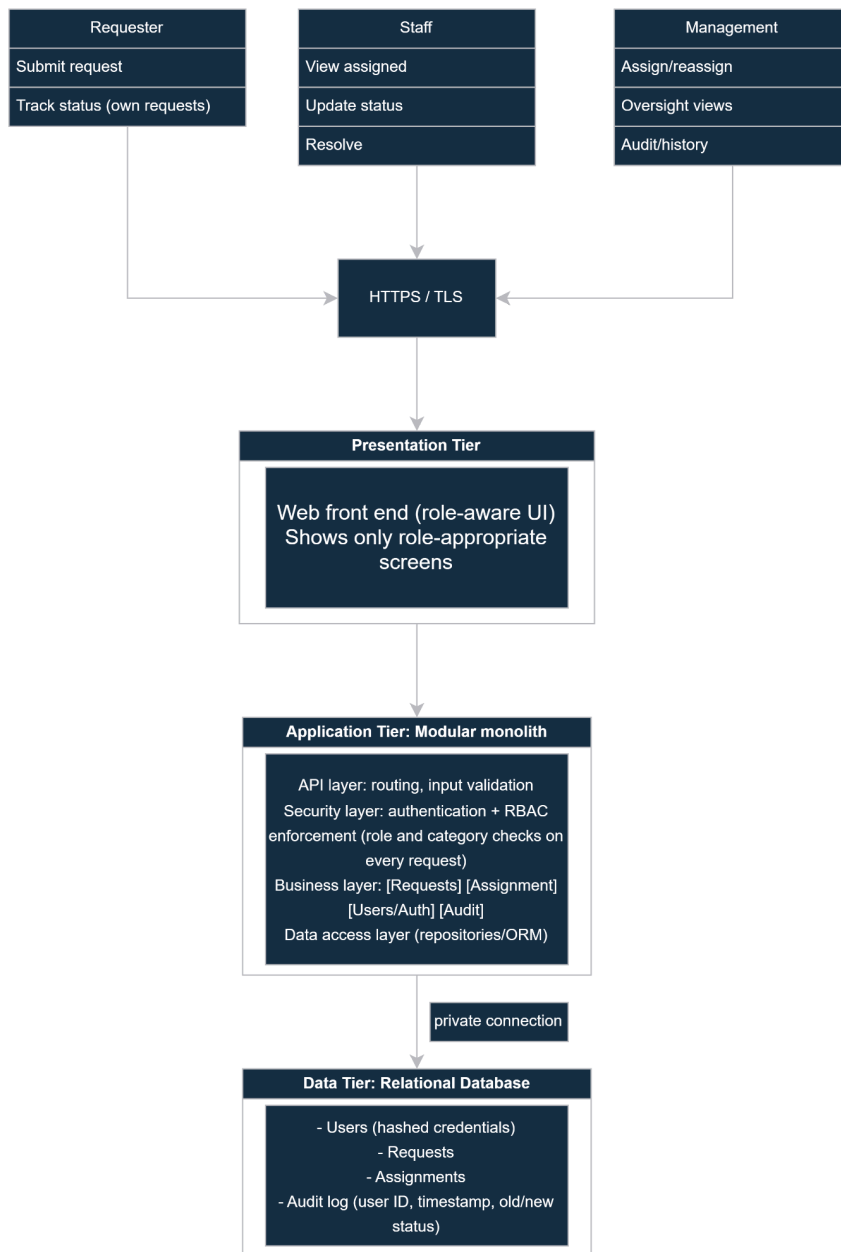
Selected architecture, justified against the ASRs above, M1 constraints, team capability, deployment implications, complexity:

Architecture Selected: Alternative A

Justification against ASRs:

- NFR-003 (RBAC): It's not just in the UI that one authentication and authorisation level is passed through at the API/service level. Role and request category access is checked first before any business logic or data access is executed. This makes the M1 test ("no exceptions") tractable, as there is a single enforcement point to test as opposed to multiple points.
- NFR-001 (Response time): With 50 concurrent users, a single well-indexed relational database and in-process module calls comfortably meet 2 seconds for 95% of actions. There are no hops across the network between services. If load increases, the stateless backend could be horizontally scaled behind a load balancer without redesign, thus avoiding the distribution cost in the meantime and solving the Forward Engineering Consideration #5.
- NFR-010 (Traceability): Status changes and their audit records are written in the same database transaction. The audit view is a simple query, and a change has to be logged in order to be successful. For a distributed design, cross-service consistency would have to be solved to ensure this.

Architecture diagram(s) - responsibilities/boundaries for the three user types (Requester/Staff/Management) and interactions:



Architecture vs. technology vs. logical layers vs. physical deployment tiers - explicitly distinguished:

- **Architecture:** How is the system organised into parts, and how do they communicate? This is the structural decision driven by the ASRs.
- **Technology:** Which concrete tools implement the architecture? These are interchangeable choices made after the architecture.
- **Logical Layers:** How is the code separated by responsibility? These are conceptual and exist within the software, regardless of where it runs.
- **Physical Development Tiers:** Where does the software run, on which machines or processes?

8) DATA AND PERSISTENCE BASELINE

8.1) Entities, ownership and lifecycle

Entities are derived from FR-001 to FR-016. The owner is the only responsibility area allowed to change that data; others read it through the owner.

Entity	Owner	Lifecycle
ServiceRequest	Request Management (status/assignment changed by Workflow)	Submitted → Assigned → In Progress → Resolved → Closed; never deleted (FR-012)
User / Role	Identity & Access	Users deactivated, never deleted, so audit references stay valid
Category	Request Management	Controlled list; retired categories deactivated
StaffCategoryScope	Identity & Access	Defines which categories each staff member may see (FR-006, NFR-003)
StatusHistory	Workflow & Assignment	Append-only audit trail (NFR-010)
RequestComment	Workflow & Assignment	Append-only feedback and resolution notes (FR-005, FR-011)

Requester contact details are personal information; no retention period is set yet, so this is raised with stakeholders (POPIA).

8.2) Initial data model

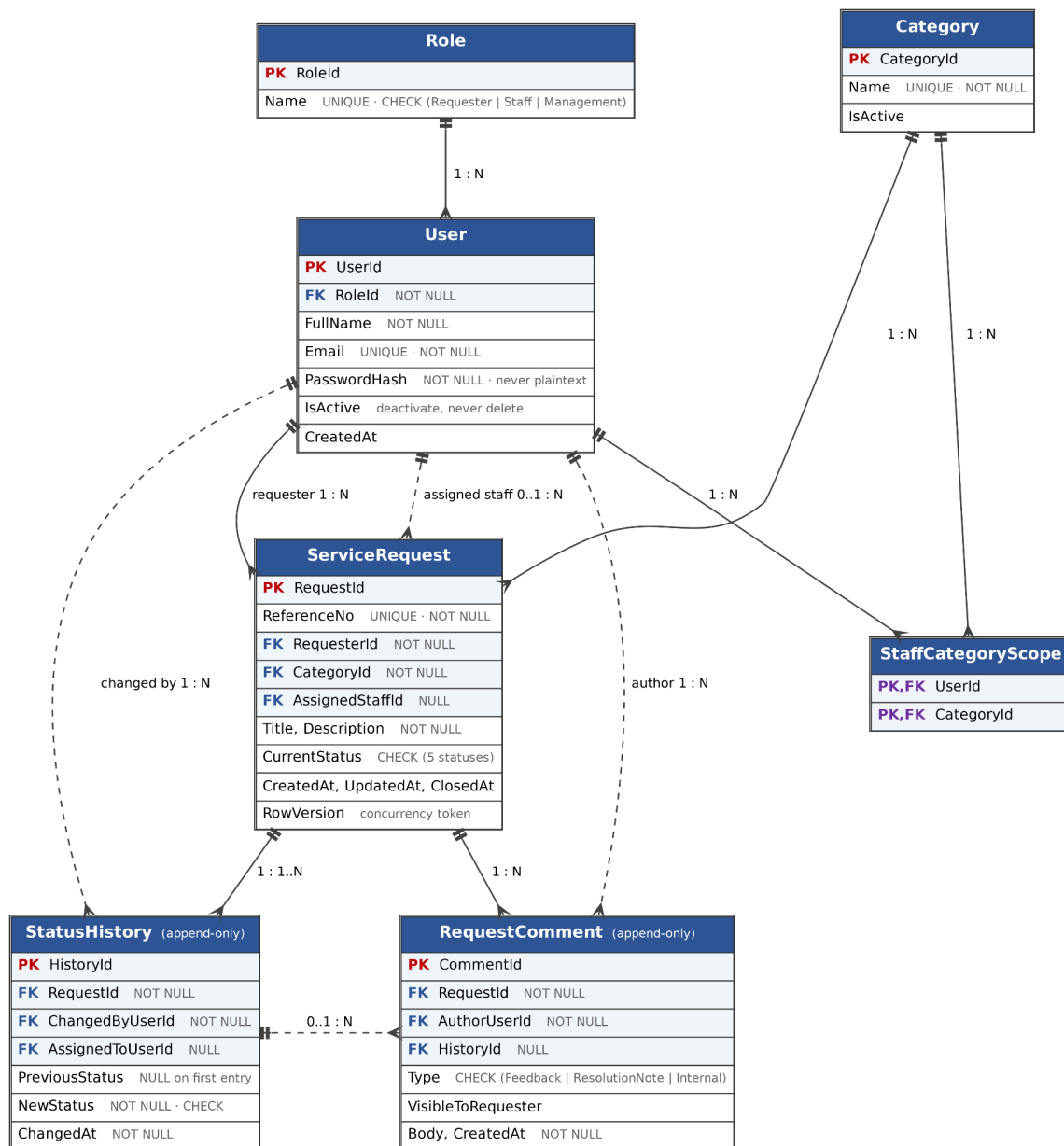


Figure 13.1 – CivicConnect initial data model

Key integrity rules: NOT NULL on mandatory request fields and a unique ReferenceNo (FR-001); FK to Category (FR-002); CHECK on the five status values (FR-003); allowed transitions enforced in the service layer; only PasswordHash stored (NFR-004).

8.3) Persistence model, SPOF and recovery

A single relational database is selected. The data is structured and strongly related, status changes and history must commit together, and management needs cross-request filtering and counts – all natural in SQL. This fits the C# decision: assignment 1 section 3 scored ASP.NET Core with Entity Framework Core (3.5 vs 3.0 for MERN), which gives migrations, transactions and concurrency tokens.

The database is a single point of failure and the main bottleneck (NFR-001, NFR-005). At the expected load (≤ 50 concurrent users) this is accepted: indexes, connection pooling and paging come first; caching and replicas are not justified without load-test evidence, and permissions must never be served stale. Recovery (NFR-008): automated daily backups stored off the host, plus a backup before every schema migration as recommended in Assignment 3 section 3.4, and a restore test before M4. Proposed targets: $RPO \leq 24$ h, $RTO \leq 4$ working hours.

8.4) Transactions, validation and concurrency

Assignment 2 section 2 compared a single synchronous transaction with an asynchronous audit write for request creation and recommended the synchronous transaction, because the async option leaves a window where a request has no audit entry (NFR-010). Applied here:

- Request creation (FR-001) and status change (FR-010) write the request and its StatusHistory row in one transaction; the service layer owns the boundary.
- Validation in three layers (Assignment 2 section 2.2): UI for usability, service layer for business rules, database constraints as the safety net.
- Assignment (FR-009) uses optimistic locking with a RowVersion check, as Assignment 2 section 2.2 recommends for this low-contention case.

9) TECHNOLOGY-STACK DECISION

Component	Decision	Evidence
Backend	ASP.NET Core (C#) .NET8	LTS releases get 3 years of support vs. 2 for STS; .NET 8 is supported until Nov 2026, the longest remaining window of any released version
Frontend	Razor Pages / MVC	Built into ASP.NET Core, no separate framework/licensing decision needed
Persistence	Entity Framework Core + SQL Server	EF Core's change-tracking supports the NFR-010 audit-trail requirement without a custom mechanism
RBAC	ASP.NET Core Identity + policy-based authorization	Built-in, directly satisfies NFR-003 without a third-party RBAC library
Licensing	MIT	Not yet independently confirmed, verify against the LICENSE file before finalizing ADR
Dependency Risk	Lower than MERN/npm, not zero	NuGet's core packages are more centrally Microsoft-maintained than

		npm's
Cost	Fully free at every layer	SDK, EF Core, SQL Server Express/SQLite, Visual Studio Community, GitHub/Actions, no paid tier required to build or run CivicConnect

10) INITIAL DESIGN DECISIONS

Design problem 1: Tightly coupled request-processing responsibilities

- A2 evidence/alternatives considered: A2 Task 1.2 compared State Pattern (groups behaviour by request state; better cohesion, but may lead to over-engineering if the workflow is simple) and Strategy Pattern (interchangeable processing rules; only useful if request types require different processing rules). A2 Task 1.3 suggested State if the issue is the status workflow and Strategy if multiple processing rules are required.
- Final M2 pattern/approach selected: State Pattern, maintained as a lightweight pattern. Each RequestState contains a RequestState, which specifies its allowed transitions, and guards. A single RequestWorkflowService processes the guard, applies the change and writes the audit record in one transaction. Strategy is not implemented.
- Why it fits CivicConnect requirements/architecture/constraints: The problem is the status workflow, as indicated in the brief. All categories have exactly one lifecycle (category is data, not behaviour), and so Strategy does not have any justification. There are role/category rules enforced in the service layer (NFR-003) and changes in status are audited in one place, the workflow service (NFR-010). The small design adapts to the scope, schedule and quality constraints, with approximately six states. It is located within the monolith, and does not have an impact on deployment.
- Expected benefit (coupling/cohesion/extensibility/maintainability/testability): Higher cohesion (SRP), no status conditionals in Request, easy extension by adding a state (OCP), transition rules in one place, and isolated unit testing of valid, invalid and unauthorised transitions.
- Complexity/trade-off introduced: Added about 6 small classes and a service rather than one status field. Modest learning curve and an over-engineering risk if the workflow doesn't change. Mitigation: Do not use a shared transition table, and do not use a generic workflow engine.
- Affected modules/components/classes/interfaces: Requests, Assignment, Audit and Security modules; Request, RequestState and its subclasses; RequestWorkflowService, TransitionGuard; IRequestRepository, IAuditLogRepository.
- If the M2 choice differs from the A2 recommendation: CivicConnect-specific evidence that changed the judgement: The M2 choice does not differ from the A2 recommendation. The controlled transitions in the brief A2 favour State over Strategy, as do the NFR-003, NFR-010 and single shared lifecycle in M2.

Design problem 2: Business logic becoming dependent on infrastructure

- A2 evidence/alternatives considered: A2 Task 1.2 compared the Adapter Pattern (isolates a specific external system, maintainability benefits, but adds a layer) with Dependency Injection (dependencies supplied rather than created. Lower coupling and better testability, but too many interfaces burden a small app). A2 Task 1.3 recommended DI generally, with Adapter only for a specific external system.
- Final M2 pattern/approach selected: IRequestRepository, IUserRepository, IAuditLogRepository, IPasswordHasher, IClock, plus a unit-of-work abstraction. Implementations are wired in at start-up through the framework's DI container. Adapter is not adopted and is reserved for any external integration approved later.
- Why it fits CivicConnect requirements/architecture/constraints: It makes the layered rule (services depend downward on abstractions) enforceable. Scope has no external integrations, so Adapter has nothing to adapt. Hashing sits behind IPasswordHasher, so the algorithm can be upgraded without touching business logic (NFR-004). Injected security and audit services cannot be bypassed and are testable with fakes (NFR-003, NFR-010). Hosting or database may change because of cost or BC Desktop compatibility, and interfaces limit that impact.
- Expected benefit (coupling/cohesion/extensibility/maintainability/testability):
- Complexity/trade-off introduced: Extra interfaces and DI configuration (the risk A2 flagged). Mitigation: interfaces only at real boundaries (persistence, auth, time). Repository abstractions can leak ORM details. Swapping the database is cheaper but not free.
- Affected modules/components/classes/interfaces: Users/Auth, Requests, Assignment, Audit, Security and Data-access layers; the interfaces above; RequestWorkflowService, AuthenticationService, AuthorisationService, AuditService and their implementations, the composition root.
- If the M2 choice differs from the A2 recommendation: The M2 does not differ from the A2 recommendation. DI is adopted as A2 recommended. Adapter is consciously not taken because the approved scope has no external integrations. Revisit through change control if email or SMS is approved.

11) INITIAL API / INTEGRATION DECISIONS

Interaction: Status change → requester feedback. Evolves the Assignment 2 section 3 interaction (Status Management → Notification), updated for the approved scope.

- **Responsibilities and information exchanged:** Producer: RequestWorkflowService in the Assignment module, after a valid, authorised status transition. Consumer: INotificationService, which records a requester-visible Feedback entry (RequestComment) shown on the requester's request view (FR-003, FR-005). Data passed: RequestId, ReferenceNo, previous status, new status, changed-at, changed-by user, optional staff comment. No requester contact details are passed.

- **Mechanism and why it is proportionate:** A synchronous in-process call through the INotificationService interface, injected via DI and executed inside the same database transaction as the status update and StatusHistory row . This differs from Assignment 2 section 3, which recommended an asynchronous event because an external email/SMS provider could slow down or fail. The approved scope has no external provider, so that reason

no longer applies; in a modular monolith a queue and worker would add a network boundary and infrastructure the Azure F1 tier does not justify.

- **Validation, failure, security and versioning:** The workflow validates the transition and the user's role/category (NFR-003) before calling the interface. Because the feedback is written in the same transaction, a failure rolls back the whole status change – a requester can never miss an update (FR-005) and the audit trail stays complete (NFR-010). The call never leaves the process, so no credentials cross it; feedback is only returned by queries scoped to the owning requester. The interface is internal and versioned with the code. If email/SMS is approved later, an adapter can consume a RequestStatusChanged event asynchronously behind the same interface (Assignment 2 section 3 Mechanism B) without changing the workflow code.

- **Web interface established so far:** GET/POST /ServiceRequests/Create and GET /ServiceRequests/Details/{id} (ASP.NET Core MVC, anti-forgery token, server-side validation), merged in PR #3. The staff status-update endpoint is Planned.

- **ADR / RTM:** RTM rows FR-005, FR-010 and NFR-005 updated.

- **Forward Engineering Consideration:** External notifications (email/SMS) are deferred. If stakeholders approve them, they go through Appendix E change control, with a new risk for provider outage.

12) DEPLOYMENT COMPATIBILITY

Concern	Finding	Status/Action
Hosting platform	Azure App Service, F1 (free) tier	Up to 10 web apps, 1 GB memory, shared compute, no cost
Capacity vs. requirements	Comfortably covers NFR-001 for academic-project scale	Performance under real load still unverified
SLA	None on F1 tier	Acceptable for staging/demo
Secrets/config	Azure App Service Application Settings	Use instead of committing connection strings/credentials to source
R-004 status	Open → Open, mitigated pending validation	Free-hosting option identified and matched to stack
Deferred	CI/CD pipeline to Azure; production-tier upgrade decision	Not required until M3

13) GITHUB GOVERNANCE, TEAM ACCOUNTABILITY & AI USAGE REGISTER

13.1) Repository governance evidence

- **Repository link:** https://github.com/AidanSm2103/SEN381_CivicConnect
- **Protected main branch:** Branch protection is enabled on main requiring a pull request before merging, with a minimum of 2 approving reviews and admin bypass disabled.
- **Required reviewer approvals:** Two approvals from members other than the author, no self-approval

13.2) Team working agreement

Enrique de Sousa: Problem Framing & Scope Lead

This role owns what CivicConnect is trying to solve and for whom, stakeholder needs, competing expectations, what falls within the scope and how schedule/cost/quality/security constraints interact with each other.

Michael-John Robinson : Requirements & Evidence Lead

This role involves turning stakeholder needs into testable commitments, writing FRs/NFRs with acceptance criteria and making sure every requirement traces back to a source.

Aidan Smith: Risk, Governance & AI Accountability Lead

This role is responsible for what could go wrong and how the team stays in control, risk identification/mitigation, thinking ahead to lifecycle concerns, repository governance and verifying AI-assisted work.

Collective responsibility is maintained through mandatory cross-review of at least one section not authored by the reviewer and all three members required as approvers on GitHub.

Communication Cadence: Brief in person meetings each day to align on progress and next steps, with the WhatsApp group used for quick questions, sharing updates and coordinating outside of those sessions.

13.3) AI Usage Register

Date	Team Member	AI Tool	Engineering task	AI Contribution	Verification Performed	Decision	Issues Found
2026/09/04	Aidan Smith	Claude	Creating the initial PED v1.0 document structure at the start of M1, based on the	Generated a section-by-section document with placeholder headings, prompts, and a proposed	Cross-checked every generated section heading and sub-heading against the actual M1	Adopted the skeleton structure as the working document framework with no content	None — structure matched the brief's required outputs exactly

			required outputs table in the Milestone 1 brief	three-way work split	brief's Required Outputs table	changes needed	
--	--	--	---	----------------------	--------------------------------	----------------	--

14) ENGINEERING DECISION LOG

Decision ID	Context	Constraints	Alternatives	Decision	Rationale	Trade-offs	Risks	Evidence	Later Consequence
D-001	The team needed a consistent, defensible method to prioritise FRs and NFRs given a fixed schedule constraint, rather than ranking requirements informally	Schedule & Quality	(a) MoSCoW (Must/Should/Could/Won't), (b) stakeholder-vote-based ranking (c) High/Medium/Low tied directly to core business needs	Adopted option (c)	Directly traceable to the stakeholder needs and business value and to the security/accountability risks already identified	Less granular than MoSCoW, so items may need review later on	Borderline requirements involve some subjective judgement; team members could disagree on edge cases	Section 3.1–3.3	To be revisited only via the formal Change Request/Impact Analysis process once the core system is stable
D-002	The team needed to select a technology stack that could support the system's functional requirements while remaining maintainable	Team familiarity; maintainability; security; integration	(a) MERN, (b) ASP.NET	Adopted option (b)	ASP.NET aligned with the team's existing C# and SQL Server experience while providing more secure packages than npm	Less access to the JavaScript/Node ecosystem than MERN; greater reliance on the Microsoft/.NET ecosystem	Framework configuration and architecture could introduce complexity	Technology-stack comparison and previous research done on both stacks	Future technology changes to be considered through the formal Change Request
D-003	A status change must give the requester feedback (FR-005) without weakening the audit trail (NFR-010)	Modular monolith (D-003); no external email/SMS in approved scope; Azure F1 hosting	(a) Synchronous in-process call in the same transaction; (b) asynchronous event + worker (Assignment 2 section 3 recommendation); (c) REST call between modules	Adopted option (a)	No external provider, so the async option's benefit does not apply; atomic with StatusHistory; no extra infrastructure	Feedback is in-app only; the status change waits for the feedback write (a local insert, negligible for NFR-001)	If email/SMS is added later, provider outages must not block status changes	Section 11; Assignment 2 section 3; Section 10	Adding external notifications requires change control, an adapter, a queue/worker and a new risk entry

					ure; testable through DI				
D-004	Requests, users, history and comments need strong integrity and a complete audit trail (NFR-010)	Schedule, Cost/Resources, Security; team building in C# with EF Core + SQL Server	(a) Relational DB, synchronous transaction; (b) document DB; (c) relational DB with asynchronous audit write	Adopted option (a)	Structured, related data; atomic audit trail; SQL reporting; strong EF Core support	Schema changes need migrations; the audit write adds slightly to each transaction (acceptable under NFR-001)	SPOF / data loss	Section 8; A2 §2; A3 §3.4; AddCategoryLookup migration (PR #3).	A move to a non-relational store would need integrity rules rewritten in code

15) PED QUALITY, DOCUMENT CONTROL & BASELINE SIGN-OFF

15.1) Document control

Version: v1.0			
Date	Reviewer	Sections Reviewed	Outcome
2026/09/30	Michael-John Robinson	Section 5,7,10	Approved
2026/09/30	Aidan Smith	Section 4,8,11	Approved
2026/09/30	Enrique de Sousa	Section 6,9,12	Approved
2026/09/30	Entire team	Full document	Approved sign-off

15.2) Baseline sign-off

Project	CivicConnect
Baseline Type	M2 - Architecture, Technology & Initial Design Baseline
Version	v2.0
Date	2026/09/30
Architecture/design reviewed	YES
Data/persistence baseline checked	YES
Technology stack decision evidenced	YES
RTM update	YES
Repository/governance controls checked	YES
Outcome	CONDITIONALLY ACCEPTED

16) REFERENCES

Pilliang, M. and Munawar, M., 2022. Risk management in software development projects: A systematic literature review. *Khazanah Informatika: Jurnal Ilmu Komputer dan Informatika*, 8(2).

Verdecchia, R., Kruchten, P., Lago, P. and Malavolta, I., 2021. Building and evaluating a theory of architectural technical debt in software-intensive systems. *Journal of Systems and Software*, 176, p.110925.